AVANCE 1 • SISTEMA FRONT-END WEB

PowerFit GYM

Guía de Preparación Técnica para la Sustentación Oral • Universidad

Stack: React 19, Vite, Tailwind CSS, Axios, React Router DOM, json-server **Operatividad:** 21 Requerimientos Funcionales (RF01–RF21)

Auditoría: Persistencia en localStorage (:3001)

🎤 Resumen Ejecutivo de 1 Minuto (Pitch de Apertura)

> *"Buenas tardes profesora. 'PowerFit GYM' es el front-end de un sistema de gestión administrativa para un gimnasio cuyo modelo de negocio no vende planes fijos, sino servicios individuales (musculación, clases dirigidas, zona húmeda, etc.) bajo suscripción mensual personalizada.*
>
> *El sistema está diseñado exclusivamente para el personal del gimnasio con roles de Administrador y Staff; los socios no inician sesión, sino que son atendidos en recepción. Está construido con React 19, Vite y Tailwind CSS, consume una API dummy en json-server mediante una instancia centralizada de Axios con interceptores automáticos de auditoría, y cuenta con un sistema de logs en localStorage.*
>
> *Implementamos 21 requerimientos funcionales y 5 reglas de negocio puras en un módulo desacoplado, garantizando validación de vigencia de acceso, descuentos automáticos y emisión de pases de invitado."*

🗺️ Mapeo Rápido: Pregunta de la Docente → Archivo a Abrir

TEMA DE LA PREGUNTA	RUTA REAL EN EL REPOSITORIO	COMPONENTE / FUNCIÓN CLAVE
Redireccionamiento y Rutas	`src/routes/AppRoutes.jsx` `src/routes/ProtectedRoute.jsx`	`ProtectedRoute`, `RoleRoute`, `<Navigate>`
Autenticación y Roles	`src/context/AuthContext.jsx` `src/pages/auth/Login.jsx`	`login()`, `useAuth()`, `powerfit_user`
Petición API e Interceptores	`src/api/axiosInstance.js` `src/api/sociosService.js`	`api.interceptors.request / response`
Auditoría y Logs	`src/utils/logger.js` `src/pages/logs/LogsPage.jsx`	`logger.info()`, `logger.warn()`, `powerfit_logs`
Macro: Contratar Membresía	`src/pages/membresias/NuevaMembresiaPage.jsx`	`handleConfirmarContratacion()`, RF14
Macro: Registrar Asistencia	`src/pages/asistencias/NuevaAsistenciaPage.jsx`	`validarMembresiaVigente()`, RF18
Reglas de Negocio Puras	`src/utils/businessRules.js`	`calcularMontoMembresia()` (R4), `calcularLimitePases()` (R1/R5)
Borrado Lógico vs Físico	`src/pages/socios/SociosPage.jsx` `src/api/asistenciasService.js`	`sociosService.deactivate()` vs `asistenciasService.delete()`

📋 Desarrollo de las 6 Preguntas de Rúbrica

1. Redireccionamiento y Control de Rutas

RUTAS Y SEGURIDAD

(A) RESPUESTA PARA DECIR EN VOZ ALTA

El enrutamiento está centralizado en `AppRoutes.jsx` con React Router DOM v7. Las rutas privadas están protegidas por `ProtectedRoute.jsx`: si no hay sesión activa en el `AuthContext`, captura la ruta intentada y usa `<Navigate to="/login" replace state={{ from: location }} />`. Si un usuario ya logueado intenta entrar a `/login`, se le redirige automáticamente a `/dashboard`. Para rutas exclusivas de administración (categorías, productos, usuarios y logs), usamos `RoleRoute.jsx`: si un usuario con rol `staff` intenta acceder, se bloquea el paso, se registra un log WARN de auditoría y se le muestra la página `/403` (Área No Autorizada). Toda ruta desconocida cae en el comodín `*` mostrando `NotFound404.jsx`.

(B) RECORRIDO PASO A PASO: USUARIO NO AUTENTICADO ENTRA A /SOCIOS

1. Usuario navega a `/socios`. `AppRoutes.jsx` detecta que está dentro del bloque `<ProtectedRoute>`.
2. `ProtectedRoute.jsx` consulta `const { isAuthenticated } = useAuth()`. Retorna `false`.
3. Registra advertencia: `logger.warn('ROUTER_PROTECTION', 'Acceso denegado', { rutaIntentada: '/socios' })`.
4. Retorna `<Navigate to="/login" state={{ from: location }} replace />`.
5. El usuario inicia sesión en `Login.jsx` y es devuelto a `location.state?.from?.pathname || '/dashboard'`.

(C) CÓDIGO PARA MOSTRAR EN PANTALLA

```
// src/routes/ProtectedRoute.jsx export default function ProtectedRoute({ children }) { const { isAuthenticated } = useAuth(); const location = useLocation(); if (!isAuthenticated) { logger.warn('ROUTER_PROTECTION', 'Acceso denegado: no autenticado', { ruta: location.pathname }); return <Navigate to="/login" state={{ from: location }} replace />; } return children; }
```

> **Pregunta de seguimiento de la docente:** "¿Por qué usas <Navigate> y no useNavigate() en ProtectedRoute?"
> *Respuesta:* "<Navigate> es declarativo y se ejecuta durante el render de React, evitando parpadeos de contenido no autorizado. `useNavigate()` es un hook imperativo para callbacks o eventos tras una acción."

2. Petición a la API y Ciclo Asíncrono (Axios)

CONSUMO API

(A) RESPUESTA PARA DECIR EN VOZ ALTA

Utilizamos **Axios** mediante una instancia centralizada en `src/api/axiosInstance.js` configurada con la URL base de json-server (`:3001`). Cuenta con interceptores de solicitud (loguea método y payload) y de respuesta (loguea status 2xx y captura errores). Aplicamos el patrón Service Layer: ningún componente llama directamente a Axios, sino a servicios especializados (como `sociosService.js`). En las pantallas, usamos `async/await` con bloques `try/catch/finally` gestionando tres estados: `loading` (spinner `<Loader />`), `error` (alerta `<ErrorMessage />` con botón de reintento) y `success` (renderizado en `<Table />` y notificación Toast).

(B) RECORRIDO PASO A PASO: LISTAR SOCIOS (GET) Y CONTRATAR MEMBRESÍA (POST)

1. **Listar Socios:** En `SociosPage.jsx`, `loadSocios()` pone `loading=true`, llama a `sociosService.getAll()` -> `api.get('/socios')` -> interceptor loguea la petición -> json-server responde 200 -> interceptor loguea respuesta -> `setSocios(data)` -> en `finally` pone `loading=false`.
2. **Contratar Membresía:** En `NuevaMembresiaPage.jsx`, al confirmar pago, `membresiasService.create()` hace un POST a `/membresias` para la cabecera y luego múltiples POST en paralelo mediante `Promise.all()` hacia `/membresia_producto` para cada servicio seleccionado.

(C) CÓDIGO PARA MOSTRAR EN PANTALLA

```
// src/api/axiosInstance.js api.interceptors.request.use((config) => { logger.info('API_REQUEST', `\${config.method.toUpperCase()} \${config.url}`, { data: config.data }); return config; }); // src/pages/socios/SociosPage.jsx const loadSocios = async () => { try { setLoading(true); setError(null); const data = await sociosService.getAll(); setSocios(data); } catch (err) { setError('Error al conectar con la API de socios.'); } finally { setLoading(false); } };
```

> **Pregunta de seguimiento de la docente:** "¿Por qué colocaste `setLoading(false)` en el bloque `finally`?"
> *Respuesta:* "Porque el bloque `finally` garantiza su ejecución tanto si la promesa termina con éxito como si falla la red, evitando que la aplicación se congele en estado de carga infinito."

3. Sistema de Auditoría y Logs

TRAZABILIDAD

(A) RESPUESTA PARA DECIR EN VOZ ALTA

El sistema de logs está centralizado en `src/utils/logger.js` y soporta niveles **INFO, WARN y ERROR**. Cada log registra un ID único, timestamp ISO, nivel, módulo emisor, acción y detalles en JSON. Se dispara automáticamente en: peticiones/respuestas de API (interceptores), login/logout, bloqueos por rol (403), validaciones de reglas de negocio y errores capturados. Los logs se imprimen con formato y color en la consola del navegador y persisten los últimos 100 registros en `localStorage` bajo la clave `powerfit_logs`. Además, creamos la vista `/logs` para el Administrador con filtros, visor de JSON formateado, botón de copiado y exportación a archivo `.json` descargable.

(B) RECORRIDO PASO A PASO: LOGS GENERADOS DURANTE EL LOGIN

1. `[INFO] [API_REQUEST] GET /usuarios`: Interceptor de salida al consultar el email ingresado.
2. `[INFO] [API_RESPONSE] GET /usuarios [200]`: Interceptor de entrada confirmando respuesta HTTP 200 de json-server.
3. `[INFO] [AUTH_LOGIN] Sesión iniciada exitosamente por Carlos Mendoza (admin)`: Registrado por `AuthContext.jsx` tras validar credenciales.
4. *(Si la contraseña hubiese sido errónea)*: Se generaría `[WARN] [AUTH_LOGIN] Contraseña incorrecta`.

(C) CÓDIGO PARA MOSTRAR EN PANTALLA

```
// src/utils/logger.js function addLog(level, modulo, accion, detalles = {}) { const logEntry = { id: Date.now() + Math.random().toString(36).substring(2, 7), timestamp: new Date().toISOString(), level, modulo, accion, detalles: typeof detalles === 'string' ? { mensaje: detalles } : detalles, }; const current = getStoredLogs(); const updated = [logEntry, ...current].slice(0, 100); // Conserva los últimos 100 localStorage.setItem('powerfit_logs', JSON.stringify(updated)); return logEntry; }
```

> **Pregunta de seguimiento de la docente:** "¿Por qué limitas el almacenamiento a los últimos 100 logs?"
> *Respuesta:* "Porque localStorage tiene una cuota máxima de 5 MB en navegadores. Limitar a los 100 eventos más recientes previene saturar el almacenamiento local sin perder la trazabilidad de las operaciones recientes."

4. Arquitectura y Estructura del Proyecto

ARQUITECTURA

(A) RESPUESTA PARA DECIR EN VOZ ALTA

La estructura en `src/` sigue una arquitectura por capas desacopladas: `api/` gestiona la comunicación HTTP con Axios y servicios de dominio; `components/common/` agrupa componentes reutilizables sin lógica de negocio (Table, Modal, FormField, Button, Loader); `components/layout/` contiene el shell visual (Navbar, Sidebar, MainLayout); `context/` administra estados globales (AuthContext y ToastContext); `hooks/` provee hooks reutilizables como useFetch; `pages/` divide las vistas por módulo; `routes/` define rutas y componentes de guarda; `utils/` reúne lógica pura (businessRules, logger, formatters); y `constants/` almacena los roles del sistema.

(B) ÁRBOL DE CARPETAS Y FLUJO

```
src/ ├─ api/ (axiosInstance.js, sociosService, membresiasService, etc.) ├─ components/ (common: Button, Table, Modal, Loader / layout: Navbar, Sidebar) ├─ context/ (AuthContext, ToastContext) ├─ hooks/ (useFetch, useAuth, useToast) ├─ pages/ (socios, membresias, asistencias, pases, usuarios, logs, etc.) ├─ routes/ (AppRoutes, ProtectedRoute, RoleRoute) └─ utils/ (businessRules.js, logger.js, formatters.js) Flujo: Página UI —> Service Layer —> Axios Instance —> json-server (:3001)
```

> **Pregunta de seguimiento de la docente:** "¿Por qué no llamaste a Axios directamente en los componentes de página?"
> *Respuesta:* "Por separación de responsabilidades y mantenibilidad. Si un endpoint cambia, se ajusta únicamente en el archivo de servicio de la entidad correspondiente sin necesidad de refactorizar múltiples vistas."

5. Lógica de la Interfaz: "Contratar Membresía" (RF14)

PANTALLA MACRO

(A) RESPUESTA PARA DECIR EN VOZ ALTA

La pantalla `NuevaMembresiaPage.jsx` ejecuta el macro flujo RF14. Al cargar, obtiene mediante `Promise.all()` los socios activos, productos y categorías. Al seleccionar un socio, consulta sus membresías previas y ejecuta la función pura `evaluarPrimeraMembresia()`: si es la primera, activa la alerta de la **Regla R5** otorgándole +1 pase bono de bienvenida. El recepcionista selecciona los servicios agrupados por categoría. En tiempo real se invoca `calcularMontoMembresia()` para aplicar el 15% de descuento si hay 4 o más productos (**Regla R4**) y `calcularLimitePases()` (**Regla R1 y R5**). Al pulsar 'Registrar Pago y Activar', guarda en `/membresias` por 1 mes y genera los detalles en `/membresia_producto`, notificando con Toast y redirigiendo al historial.

(C) CÓDIGO PARA MOSTRAR EN PANTALLA

```
// src/pages/membresias/NuevaMembresiaPage.jsx const selectedProducts = productos.filter((p) =>
selectedProductIds.includes(p.id)); // Cálculos puros automáticos const financialSummary =
calcularMontoMembresia(selectedProducts); // Regla R4 const pasesSummary = calcularLimitePases(selectedProducts,
isPrimeraMembresia); // Reglas R1 y R5
```

> **Pregunta de seguimiento de la docente:** "¿Cómo previenes que se registre una membresía sin productos o sin socio?"
> *Respuesta:* "El botón de confirmación se deshabilita condicionalmente mediante `disabled={!selectedSocioId || selectedProducts.length === 0}` y además la función de guardado implementa cláusulas de guarda con alertas Toast defensivas."

6. Regla de Negocio: R4 (Descuento 15% por 4+ Productos)

REGLA DE NEGOCIO

(A) RESPUESTA PARA DECIR EN VOZ ALTA

La **Regla R4** establece que si una membresía incluye 4 o más productos contratados, el `monto_total` es igual a la suma de precios base con un 15% de descuento (`suma * 0.85`). Si tiene menos de 4 productos, se cobra la suma sin descuento. Está implementada como función pura en `src/utils/businessRules.js` bajo el nombre `calcularMontoMembresia()`. La pantalla `NuevaMembresiaPage.jsx` la consume en vivo, mostrando al usuario el subtotal, el ahorro monetario exacto y el total final antes de efectuar el pago.

(B) EJEMPLOS NUMÉRICOS DE DEMOSTRACIÓN

- **Cumplimiento (4 productos):** Musculación (S/. 120) + Spinning (S/. 80) + Sauna (S/. 65) + Casillero (S/. 35) = Subtotal S/. 300.00. Aplica 15% de descuento (-S/. 45.00). **Total final = S/. 255.00**. Mensaje: "*¡Descuento del 15% aplicado por contratar 4 productos! (Regla R4)*".
- **Incumplimiento (3 productos):** Musculación (S/. 120) + Spinning (S/. 80) + Sauna (S/. 65) = Subtotal S/. 265.00. Descuento 0%. **Total final = S/. 265.00**. Mensaje: "*Agrega 1 producto(s) más para acceder al 15% de descuento por volumen (Regla R4)*".

(C) CÓDIGO PARA MOSTRAR EN PANTALLA

```
// src/utils/businessRules.js (Regla R4) export function calcularMontoMembresia(productos = []) { const subtotal =
productos.reduce((acc, p) => acc + (Number(p.precio_base) || 0), 0); const aplicaDescuento = productos.length >= 4;
const montoDescuento = aplicaDescuento ? subtotal * 0.15 : 0; const montoTotal = subtotal - montoDescuento; return {
subtotal: Number(subtotal.toFixed(2)), montoDescuento: Number(montoDescuento.toFixed(2)), montoTotal:
Number(montoTotal.toFixed(2)), aplicaDescuento, }; }
```

❓ 10 Preguntas Extra Típicas de Sustentación

PREGUNTA DE LA DOCENTE	RESPUESTA TÉCNICA SUGERIDA
1. ¿Por qué Context API y no Redux?	Para este Avance 1, solo la sesión (`AuthContext`) y las notificaciones (`ToastContext`) son globales. Context API evita dependencias pesadas y resuelve el problema con herramientas estándar de React.
2. ¿Qué pasa si json-server está apagado?	El interceptor de Axios detecta `ERR_NETWORK`, genera un log `ERROR` y la pantalla activa el componente `<ErrorMessage>` con un botón para reintentar la conexión.
3. ¿Por qué borrado lógico en socios y físico en asistencias?	El socio tiene historial referencial (Regla R2: asistencias y membresías no deben perderse). En asistencias se permite borrado físico (RF21) para corregir marcaciones erróneas del molinete.
4. ¿Por qué funciones puras en businessRules.js?	Porque no tienen efectos secundarios ni dependen del DOM o de APIs, lo que permite testearlas de forma unitaria y reutilizarlas en múltiples componentes sin acoplamientos.
5. ¿Cómo persiste la información al presionar F5?	La sesión se lee de `localStorage (powerfit_user)`, los logs de `powerfit_logs`, y los datos del gimnasio se vuelven a solicitar a json-server que los tiene guardados en `db.json`.
6. ¿Cómo se valida la asistencia de un socio?	En `NuevaAsistenciaPage.jsx`, `validarMembresiaVigente()` verifica que el socio tenga una membresía activa cuya `fecha_fin >= hoy`. Si venció, bloquea el registro con tarjeta roja.
7. ¿Cómo funciona la Regla R5 en socios antiguos?	Al consultar `/membresias?socio_id=X`, si ya tiene registros previos, `esPrimera` es `false` y no recibe el bono +1, rigiéndose solo por el cálculo base de R1.
8. ¿Cómo garantizas la accesibilidad (a11y)?	En `Modal.jsx` escuchamos la tecla `Escape`, los inputs en `FormField.jsx` asocian `htmlFor` con `id`, y los botones poseen contrastes adecuados y estados focus visibles.
9. ¿Cómo manejas el diseño responsive?	Con Tailwind CSS y enfoque Mobile-First. En pantallas pequeñas el menú lateral se oculta y se despliega con botón hamburguesa, y las tablas poseen contenedores con scroll horizontal.
10. ¿Por qué elegiste lucide-react para los iconos?	Porque provee iconos SVG ligeros compatibles con tree-shaking, importando solo los necesarios para mantener optimizado el tamaño del bundle en producción.

🛡️ Puntos Débiles del Proyecto y Cómo Defenderlos con Honestidad

1. "Las contraseñas están en texto plano en db.json y no hay JWT real"

Cómo responder: "*Efectivamente profesora. Para el alcance de este Avance 1, la rúbrica solicitó expresamente una API dummy con json-server y simulación de sesión en localStorage. En un entorno de producción, las contraseñas se almacenarían con hash bcrypt en una base de datos real y la autenticación se gestionaría mediante tokens JWT en cookies HttpOnly seguras.*"

2. "La paginación y búsqueda de la tabla se ejecutan en memoria y no en el servidor"

Cómo responder: "*Es correcto. Para el volumen de datos de este avance en un prototipo local, el filtrado en cliente es instantáneo y ahorra llamadas de red. Sin embargo, nuestro servicio `sociosService.js` está diseñado para soportar en el siguiente avance los parámetros `?_page=1&_limit=10` de paginación del lado del servidor.*"

3. "No existe sincronización en tiempo real con WebSockets entre dos recepcionistas"

Cómo responder: "*Así es. Para este primer avance no se contempló arquitectura reactiva con WebSockets. Sin embargo, la integridad se mantiene porque cada acción (como registrar asistencia o crear membresía) ejecuta una petición HTTP que actualiza inmediatamente `db.json` y refresca el estado local.*"